AIA13: Edge AI and Optimization

Module Introduction and Theoretical Foundations

Edge AI deploys **ML models on resource-constrained devices** (IoT, mobile, embedded), enabling real-time inference with **latency < 100ms**, **memory < 100MB**, and **battery efficiency**. This module covers **quantization**, **pruning**, **NAS**, **knowledge distillation**, and **hardware-aware optimization**.

Optimization Goals:

text

Accuracy: ≥ 95% of full-precision

Latency: < 50ms on target device

Memory: < 256MB

Power: < 1W continuous

Theoretical Framework:

1. **Quantization Error Bounds**
2. **Lottery Ticket Hypothesis** (pruning)
3. **Distillation Divergence**
4. **Hardware Performance Counters**

(Figure 1: Model size vs accuracy vs latency Pareto frontier)

Section 1: Quantization - From FP32 to INT4

Quantization Theory:

text

r = clamp((x - z) / s, q_min, q_max)

x = s(r + z) + noise

MSE minimized by min-max or percentile

Quantization Types:

1. **Post-Training Quantization (PTQ):**

text

Static: Calibrate with 128-1024 samples

Dynamic: Runtime per-tensor scale

1. **Quantization-Aware Training (QAT):**

text

Straight-Through Estimator (STE):

∂r/∂x ≈ 1 during backprop

Complete Quantization Pipeline:

python

```python
import torch
import torch.quantization as quant
from torch.quantization import quantize_dynamic

class QuantizationPipeline:
    def __init__(self, model):
        self.model = model
        self.calibration_data = self.prepare_calibration()

    def ptq_static(self, dtype=torch.qint8):
        """Post-training static quantization"""
        quantized_model = quant.quantize_dynamic(
            self.model, {torch.nn.Linear}, dtype=torch.qint8
        )
        return quantized_model

    def ptq_dynamic(self):
        """Dynamic per-output quantization"""
        return quantize_dynamic(self.model, {torch.nn.Linear}, dtype=torch.float16)

    def qat(self, train_loader, epochs=10):
        """Quantization-aware training"""
        self.model.qconfig = torch.quantization.get_default_qat_qconfig('fbgemm')
        self.model_prepared = torch.quantization.prepare_qat(self.model, inplace=True)

        # Train with QAT
        for epoch in range(epochs):
            self.train_step(train_loader)
```

```
        # Convert
        self.model_quantized = torch.quantization.convert(
            self.model_prepared, inplace=True
        )
        return self.model_quantized

    def calibrate(self, model, calibration_loader):
        """FX Graph Mode calibration"""
        model.eval()
        model.qconfig = torch.quantization.default_qconfig
        model_prepared = torch.quantization.prepare(model, inplace=False)

        with torch.no_grad():
            for batch in calibration_loader:
                model_prepared(batch)

        model_quantized = torch.quantization.convert(model_prepared, inplace=False)
        return model_quantized
```

INT4 Quantization (QLoRA-style):

text

NF4: NormalFloat4 distribution

Double quantization: Quantize scales

Section 2: Pruning - Structured and Unstructured

Lottery Ticket Hypothesis (Frankle 2018):

text

Random subnetwork matches full training

Iterative Magnitude Pruning (IMP) finds it

Pruning Methods:

text

1. Magnitude: Remove smallest weights

2. Gradient: Taylor expansion 1st/2nd order

3. Movement: Optimal Brain Surgeon

4. Structured: Filter/Neuron/Channel pruning

Complete Pruning Pipeline:

python

```python
import torch.nn.utils.prune as prune
from torch.nn.utils import prune

class PruningPipeline:
    def __init__(self, model, sparsity=0.9):
        self.model = model
        self.sparsity = sparsity

    def unstructured_magnitude(self, amount=0.9):
        """Global unstructured pruning"""
        parameters_to_prune = []
        for name, module in self.model.named_modules():
            if isinstance(module, torch.nn.Linear):
                parameters_to_prune.append((module, 'weight'))

        prune.global_unstructured(
            parameters_to_prune,
            pruning_method=prune.L1Unstructured,
            amount=self.sparsity
        )

        # Make permanent
        for module, param in parameters_to_prune:
            prune.remove(module, param)
```

```
    return self.model

def structured_channel(self, amount=0.5):
    """Channel pruning (Conv2d)"""
    for name, module in self.model.named_modules():
        if isinstance(module, torch.nn.Conv2d):
            prune.ln_structured(module, name='weight', amount=amount,
                                n=module.out_channels, dim=0)
    return self.model

def lottery_ticket(self, iterations=20, prune_rate=0.2):
    """Iterative Magnitude Pruning"""
    base_weights = {name: param.clone() for name, param
                    in self.model.named_parameters()}

    for i in range(iterations):
        # Reset weights
        for name, module in self.model.named_modules():
            if hasattr(module, 'weight_mask'):
                prune.remove(module, 'weight')

        self.model.load_state_dict(base_weights)

        # Train 1 epoch
        self.train_epoch()

        # Prune
        prune.global_unstructured(self.model, pruning_method=prune.L1Unstructured,
                                  amount=prune_rate)

    return self.model
```

Section 3: Knowledge Distillation

Distillation Theory (Hinton 2015):

text

Student learns soft logits from Teacher

KL(σ_T || σ_S / T)

T: Temperature (softens distributions)

Distillation Implementation:

python

```python
class DistillationTrainer:
    def __init__(self, teacher, student, temperature=4.0, alpha=0.9):
        self.teacher = teacher
        self.student = student
        self.T = temperature
        self.alpha = alpha  # KL weight

    def distillation_loss(self, x, y):
        # Student logits
        s_logits = self.student(x)
        s_soft = F.log_softmax(s_logits / self.T, dim=1)

        # Teacher soft labels
        with torch.no_grad():
            t_logits = self.teacher(x)
            t_soft = F.softmax(t_logits / self.T, dim=1)

        # Distillation loss
        dist_loss = F.kl_div(s_soft, t_soft, reduction='batchmean') * (self.T ** 2)

        # Hard labels
        ce_loss = F.cross_entropy(s_logits, y)
```

```
    return self.alpha * dist_loss + (1 - self.alpha) * ce_loss
```

Section 4: Neural Architecture Search (NAS) for Edge

Differentiable NAS (DARTS):

text

α_i: Architecture parameters

softmax(α): Operation weights

MobileNetV3-style Manual Search:

text

Repeat: Inverted Residual + H-Swish

NAS Cell: 5-7 operations per edge

Section 5: Hardware-Specific Optimization

TensorRT Optimization:

text

1. Layer fusion (Conv+BN+ReLU)
2. Kernel auto-tuning
3. INT8 calibration

ONNX Runtime:

python

```
import onnxruntime as ort

# Export to ONNX
torch.onnx.export(model, dummy_input, "model.onnx",
                  opset_version=17, do_constant_folding=True)

# Optimize
from onnxruntime import quantization
quantized_model = quantization.quantize_dynamic("model.onnx", "model_quant.onnx")
```

```
# Inference
session = ort.InferenceSession("model_quant.onnx",
                               providers=['CUDAExecutionProvider'])
```

TVM (Apache TVM):

```
text
Hardware-specific code generation
End-to-end from PyTorch → ARM/Edge TPU
```

Section 6: Edge Deployment Patterns

Mobile (TensorFlow Lite):

```
python
import tensorflow as tf

# Convert
converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

# Delegate to GPU/NPU
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS,
    tf.lite.OpsSet.SELECT_TF_OPS
]
```

Embedded (TinyML):

```
text
uTensor, CMSIS-NN for ARM Cortex-M
Binary Neural Networks (<1MB models)
```

Federated Learning:

```
text
On-device training with DP-SGD
FedAvg aggregation
```

Section 7: Production Edge AI Systems

Benchmarking Framework:

python

```python
class EdgeBenchmark:
    def __init__(self, device):
        self.device = device

    def measure_latency(self, model, inputs, warmup=10, iters=100):
        model.eval()
        with torch.no_grad():
            # Warmup
            for _ in range(warmup):
                _ = model(inputs)

            # Benchmark
            start = torch.cuda.Event(enable_timing=True)
            end = torch.cuda.Event(enable_timing=True)

            timings = []
            for _ in range(iters):
                start.record()
                _ = model(inputs)
                end.record()
                torch.cuda.synchronize()
                timings.append(start.elapsed_time(end))

        return np.mean(timings)
```

Case Studies:

1. **Apple CoreML**: iOS Neural Engine
2. **Google MediaPipe**: Real-time ML pipeline

3. **Tesla FSD**: 200+ TOPS inference

Capstone: Edge Computer Vision System

Object detection → pose estimation → action recognition

< 30ms on Snapdragon 888